



C Notes by Coding Age

Mastering C Programming – From Basics to Advanced

INDEX

1. Introduction to Programming	5
1.1 What is a Programming Language?	5
1.2 Introduction to C Language	5
1.3 Compiling and Running a Program	5
1.4 First Program in C	6
2. Variables in C.....	7
2.1 What is a Variable?	7
2.2 Variable Naming Rules	7
3. Data Types in C.....	8
3.1 Types of Data	8
3.2 Format Specifiers	9
3.3 Signed vs Unsigned Data Types	9

4. Keywords in C 10

4.1 Keywords Overview 10

5. Input/Output in C 11

5.1 Taking Input: `scanf()` 12

5.2 Displaying Output in C: `printf()` 13

6. Operators in C 15

6.1 Unary Operators 16

6.2 Binary Operators (Arithmetic, Relational, Logical, Bitwise) 18

6.3 Assignment Operators 22

6.4 Ternary Operator 23

7. Conditional Statements in C 24

7.1 `if` , `if-else` , `else-if` , Nested `if-else` 25

7.2 Nested `if-else` 26

7.3 `switch` Statement 27

8. Loops in C 29

8.1 `while` Loop 30

8.2 `for` Loop 31

8.3 `do-while` Loop 32

8.4 Nested Loops 33

8.5 `break` and `continue` Statements 34

9. Functions in C 35

9.1 Types: Return vs Non-return 37

9.2 Parameters and Arguments 39

9.3 Calling Functions 40

10. Arrays in C	42
10.1 One-Dimensional Arrays (<code>1D</code>)	42
10.2 Two-Dimensional Arrays (<code>2D</code>)	46
11. Strings in C	50
11.1 Declaration and Initialization	50
11.2 String Input/Output	51
11.3 String Functions (<code>strlen</code> , <code>strcpy</code> , etc.)	52
12. Pointers in C	54
12.1 Pointer Basics	54
12.2 Pointer to Array and Strings	56
12.3 Pointers with Functions	57
12.4 Double Pointer	58
13. Structures and Unions	59
13.1 Structures: Syntax and Examples	59
13.2 Accessing Members and Initialization	60
13.3 Structure with Pointers	61
13.4 <code>typedef</code> with Structure	62
13.5 Array of Structures	63
13.6 Nested Structures	63
13.7 Union: Syntax and Use	64
13.8 Structure vs Union	65
14. Dynamic Memory Allocation (DMA)	65
14.1 Static vs Dynamic Allocation	65
14.2 <code>malloc()</code> , <code>calloc()</code> , <code>realloc()</code> , <code>free()</code>	66
14.3 2D Array Using DMA.....	71
15. File Handling in C	73

15.1 What is a File?	74
15.2 File Modes	74
15.3 Opening and Closing Files	75
15.4 Reading and Writing Files	76
15.5 Append Mode	76
15.6 File Functions: <code>fseek()</code> , <code>ftell()</code> , <code>rewind()</code>	78

Programming Language :

A **programming language** is a set of vocabulary and grammar rules used to write instructions that a computer can understand and execute. It acts as a bridge between humans and machines, allowing us to communicate with computers to create software, applications, websites, and more.

C Language:

- **C is a Procedural Programming Language**, which means it follows a step-by-step approach to solve a problem.
 - In a procedural language, the program is executed in a sequence of instructions or commands to produce the desired output.
 - The entire program is divided into smaller blocks called **functions** or **procedures**, making the code modular, easier to manage, and understand.
 - **C is a statically typed language**, meaning variables must be declared with a data type, and their types are checked at **compile time**.
-

Compiling a Program:

- The code we write, known as **source code**, is written in a human-readable form. However, computers can only understand **binary machine code** (i.e., 0s and 1s).
- **Compiling** is the process of converting this source code into **machine code**, so that the computer can understand and execute it.
- In C, we typically use the `gcc` (GNU Compiler Collection) command to compile the code.

For example:

```
gcc filename.c
```

This command generates an executable file, usually named `a.exe` (on Windows).

- Every time we make changes to the source code, we must **recompile** the program. This ensures that the latest changes are reflected in the new executable file.

Running a program(execution) :

- The process when a program performs the given task after it has been compiled, is known as running or execution.
- To run a program after it has been compiled, we use the command **a.exe**

▼ Below we have our first code in C language :

Example 1

```
#include <stdio.h>
int main(){
printf("Hello World!");
return 0;
}
```

Output: Hello world!

Explanation of Each Line:

- `#include <stdio.h>` — This is a **preprocessor directive** that tells the compiler to include the **standard input-output header file**.
(Header files provide pre-defined functions that make programming easier.)
- `int main() { }` — This defines the **main function**, the starting point of every C program.
- `printf("Hello World!");` — This is a **statement** that prints the text `"Hello World!"` on the screen.
- `return 0;` — This line returns the value 0 to the operating system, indicating the program ended successfully.
- The **semicolon (;)** at the end of each statement marks the end of that instruction.

- To save a C program file, use the extension `.c`

Example: `file.c` where `file` is the filename.

Example 2 :

```
#include <stdio.h>
int main(){
int n = 5;
char b = 's';
return 0;
}
```

In the above code, we have two lines that are new to us :

First is, `int n = 5;`

- - Here, n is a **variable** and the word `'int'` written right before it, is the **datatype** of variable n.
- - 5 is the value that has been assigned to the variable n.
- - This is how we declare a variable in C.
- - Assigning value to a variable is called initialization.

second is, `char b = 's';`

- - This statement is similar to the first one. The only difference is that the datatype of variable b is char.
- - The value assigned to variable b is 's'(a letter or character).

Variable :

- A **variable** is a name given to a specific memory location that stores a value.
- It acts as a container that holds data which can be changed or used throughout the program.
- In the previous example, `n` and `b` are two different variables, each storing their own values.

Variable naming rules :

- Variable names can consist only of **alphabets, numbers, underscore** (`'_'`) or **dollar sign** (`'$'`).
- No other special character is allowed. Space is also not allowed.
- The first character in the variable name must be an alphabet or underscore.
- Variables are Case-Sensitive.



ex: `int a;` `int a;` **✗** //not allowed

ex: `int a;` `int b;` **✓** //allowed

- The length of a variable name should not be more than **31 characters**.

Data type :

- Data type is classification of data.
- As the name suggests, data type tells the type of information that can be stored in a given variable.
- Data can be classified into any of the following data types in C :

1. `int`
2. `long`
3. `float`
4. `double`
5. `char`

(These are only the frequently used ones, there are a few more data types in C.)

Data type	Size	Description	Example
-----------	------	-------------	---------

int	2 or 4 bytes	Stores only an integer	int n = 1;
long	8 bytes	Stores integer but range of values is greater than int data type	long k = 23125;
float	4 bytes	Stores decimal numbers. Sufficient for storing up to 6-7 decimal digits	float f = 2.5;
double	8 bytes	Stores decimal numbers. Sufficient for storing up to 15 decimal digits	double d = 3.5;
char	1 bytes	Stores a single character/letter/number, within single quotes.	char m = 't';

Note: Size and range of a data type may vary depending on the system being used. The sizes mentioned above are the general ones.

Format Specifier in C

What is it?

A **format specifier** tells the compiler **what type of data** is being input or output. It is used inside functions like `scanf()` and `printf()` to properly **read or display values**.

✓ It **always starts with a % symbol**, followed by a character that represents the **data type**.

- Every data type has its own format specifier, below is the list of format specifiers of some commonly used data types:

Common Format Specifiers

Data Type	Format Specifier	Used With
int	%d	Integer values
long	%ld	Long integers
float	%f	Floating-point numbers
double	%lf	Double-precision floats
char	%c	Single characters

Signed and Unsigned Data Types: Range Explained

1. Signed Data Types (using 2's complement representation)

- **Range:** $-2(n-1)$ to $2(n-1) - 1$
- **where, n is the number of bits in that data type. (1 byte = 8 bits)**
- **Explanation:**
 - The first bit is used for sign (0 = positive, 1 = negative).
 - So, you have one less bit for magnitude $\rightarrow n - 1$.
 - Negative range includes $-2(n-1)$ (minimum).
 - Positive range goes up to $2(n-1)$ (maximum).

2. Unsigned Data Types (only positive values and zero)

- **Range:** 0 to $2(n - 1)$
- **Explanation:**
 - All bits are used for the magnitude (no sign bit).
 - So values start at 0 and go up to the maximum number represented by all bits set to 1.

Keywords :

- Keywords are the words that have meanings already assigned to them by the language and they're fixed.
- They cannot be used as variable, function names, or identifiers.
- Every keyword has its own job which makes code writing easier for us.
- They have specific meanings and roles in the language.
- Below is the complete list of all 32 keywords available in C :

int	if	return	auto
long	else	register	void

float	break	signed	unsigned
double	continue	sizeof	const
char	default	static	extern
short	do	struct	goto
switch	while	typedef	enum
case	for	union	volatile

Taking Input from the User in C

In C programming, we take input from the user using a special function:

scanf() – *Function to Take Input*

The `scanf()` function allows users to **enter values** from the keyboard, which are then stored in **variables**.

Syntax of `scanf()` :

```
scanf("format specifier", &variable_name);
```

Inside `scanf()` :

1. **Format Specifier** : A formatted string (which basically means, format specifier inside double quotes) e.g. :
`"%d"`. It tells the system, the type of value that has to be taken as input.
2. Address of the variable in which we want to store the user input (attaching `'&'` to the left of variable name gives the address of that variable) e.g. :
`&v`
3. Format specifier and variable address are separated by a **comma** `,`.

Example 3

```
#include <stdio.h>

int main() {
    int v, m, n;
    float b;

    scanf("%d", &v);    // taking an integer input
    printf("%d\n", v);  // displaying the integer

    scanf("%f", &b);    // taking a float input
    printf("%f\n", b);  // displaying the float

    scanf("%d%d", &m, &n); // taking two integers
    printf("%d and %d\n", m, n); // displaying both integers

    return 0;
}
```

What Happens When You Run This?

1. The program asks for **1 integer** – stores it in `v` and displays it.
2. Then it asks for **1 float** – stores it in `b` and displays it.
3. Then it asks for **2 integers** – stores them in `m` and `n`, then prints them with "and" in between.

Things that are new to us in the above code :

`scanf()` → function to take user input

`printf()` → function to give output

*(**printf() and scanf() are two of the many functions defined in stdio.h header file)*

`"%d", "%f"` → format specifier

`"&v", &m, &n, &b` → address of variable (location where a variable is located in the memory)

New Concepts in This Code

Concept	Description
---------	-------------

<code>scanf()</code>	Used to take input from the user
<code>printf()</code>	Used to display output
<code>%d</code> , <code>%f</code>	Format specifiers <code>%d</code> for integer, <code>%f</code> for float
<code>&v</code> , <code>&b</code>	Address of variables , where the input should be stored

 Both `scanf()` and `printf()` are two of the many functions defined in the `stdio.h` header file, which must be included at the top of your program.

Tip:

- You **must** use `&` before variable names in `scanf()` (but **not** in `printf()`).
- Multiple inputs can be taken in a single `scanf()` by adding more format specifiers and variables, like this:

```
scanf("%d %f", &a,&b);
```

Displaying Output in C

In C programming, we display output to the user using a special function:

`printf()` – *Function to Display Output*

The `printf()` function **prints text or variable values** to the screen. It's one of the most commonly used functions in C.

Syntax of `printf()` :

```
// 1. Print a simple message or string
printf("Your message here");

// 2. Print formatted output using format specifiers
printf("Format specifier", variable_name);
```

Inside `printf()` :

1. **Format Specifier:** Written inside **double quotes**. It tells the program **what type of data** to print — like `%d` for integers or `%f` for float.

2. **Variable Name:** Placed **after the comma**, outside the quotes — it's the actual value that we want to print.
3. Both the format specifier and the variable name are **separated by a comma**.

Example: Displaying Output

```
#include <stdio.h>

int main() {
    int age = 20;
    float weight = 55.5;

    printf("My age is %d years.\n", age);    // prints: My age is 20 years.
    printf("My weight is %.1f kg.\n", weight); // prints: My weight is 55.5 kg.

    return 0;
}
```

What Happens When You Run This?

- `printf()` prints the message inside quotes and replaces `%d` and `%f` with the values of `age` and `weight`.
- `\n` is a **newline character**, which moves the output to the next line after printing.

Example: Combining Input and Output

```
#include <stdio.h>

int main() {
    int num;
    float price;

    printf("Enter a number and a price:\n");
    scanf("%d %f", &num, &price); // taking input
```

```
printf("You entered number = %d and price = %.2f\n", num, price); // output  
  
return 0;
```

 Output (if user enters `10 45.67`):

```
You entered number = 10 and price = 45.6
```

Tips:

- You **do not use** `&` in `printf()` — only use it in `scanf()`.
- You can print **multiple variables** by using multiple format specifiers:

```
printf("a = %d, b = %d", a, b);
```

- Use `\n` to **neatly format** your output on separate lines.

Operators :

What is an Operator?

An operator is a symbol or sign that tells the computer to perform a specific action or calculation on one or more operands.

What are Operands?

Operands are the values or data on which the operator acts. Think of operands as the “ingredients” and operators as the “action” that combines or changes those ingredients.

Example to Understand:

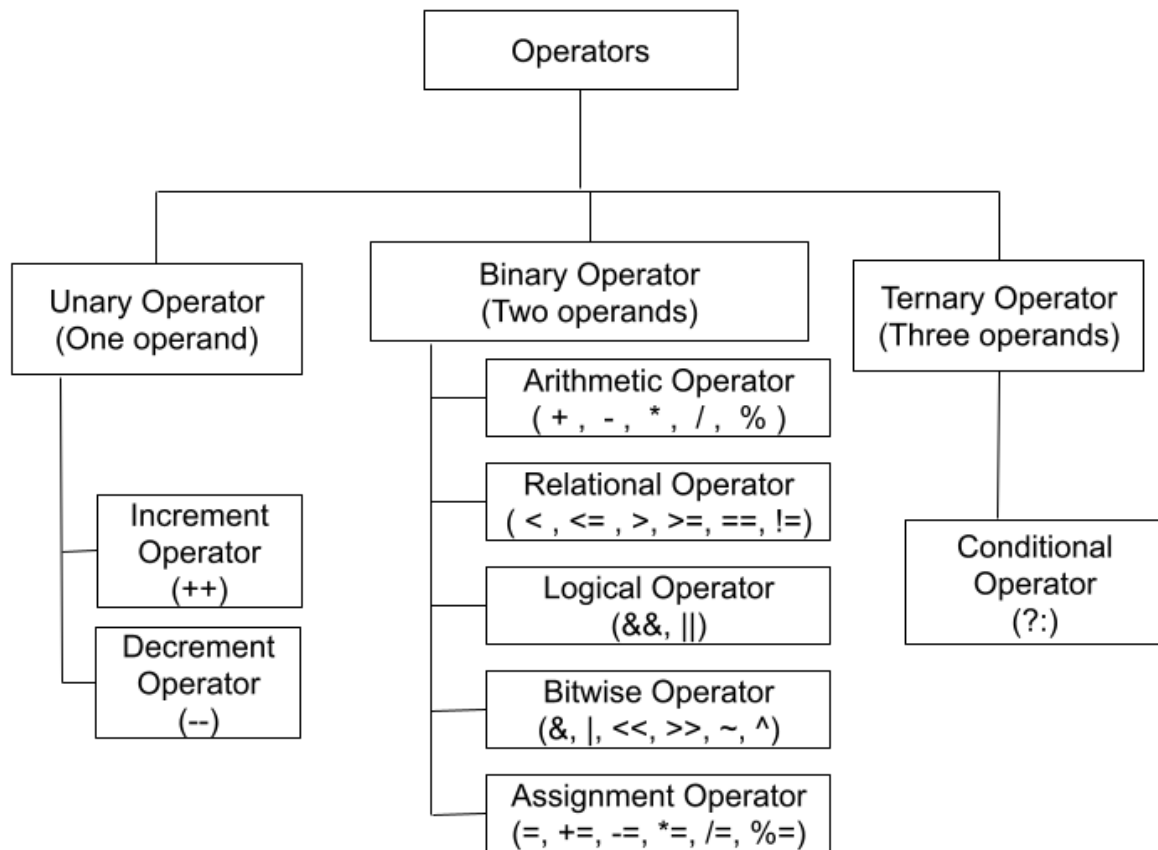
If you see this:

```
2 + 3
```

- The `+` is the **operator** — it means **add**.
- The numbers `2` and `3` are the **operands** — the values to be added.

The computer reads this as:

"Add 2 and 3 together" → Result is 5.



1. Unary Operator:

1) Increment Operator :

Pre-Increment (`++var`)

- The `++` operator is placed **before** the variable (on the immediate left).
- **First**, it increases the variable's value by 1.
- **Then**, it uses the updated value in the expression.

Post-Increment (`var++`)

- The `++` operator is placed **after** the variable (on the immediate right).
- **First**, it uses the current value of the variable in the expression.

- **Then**, it increases the variable's value by 1 for future use.

2) Decrement Operator :

Pre-Decrement (**-var**)

- The **-** is placed **before** the variable.
- **First**, it decreases the value by 1.
- **Then**, it uses the updated value in the expression.

Post-Decrement (**var--**)

- The **-** is placed **after** the variable.
- **First**, it uses the current value in the expression.
- **Then**, it decreases the value by 1 for future use.

Example 4 :

```
#include <stdio.h>
int main(){
    int a = 4, b = 10;
    int z = ++a; //Pre Increment
    printf("%d ", z); // Output is 5 (increased immediately)
    int y = b++; //Post Increment
    printf("%d ", y);
    // Output is 10 (did not increase in the current expression)
    y = b; //now increased by 1
    printf("%d ", y); // Output is 11
    //-----Same goes for Decrement Operator,
    it decreases by 1
    int c = 21, d = 7;
    int x = --c; //Pre Decrement
    printf("%d ", x); // Output is 20 (decreased immediately)
    int w = d--; //Post decrement
    printf("%d ", w); // Output is 7 (did not decrease in the current expression)
    w = d; //now decreased by 1
```

```
printf("%d ", w); // Output is 6
return 0;
}
```

✅ **Tip to Remember:**

- **Pre** = Change first, then use
- **Post** = Use first, then change

2. Binary Operator:

A

binary operator works with **two operands**. It performs operations using one operator and two values.

1) Arithmetic Operator(+, -, *, /, %) :

- Performs basic mathematical operations.
- Gives numeric value as an output.

Example 5 :

```
#include <stdio.h>
int main(){
int a = 3;
int b = 2;
printf("%d", (a+b)); // Output is 5
return 0;
}
```

2) Relational Operator(<, <=, >, >=, ==, !=) :

- Compares the operands.
- Gives output as either true(1) or false(0).

Example 6:

```
#include <stdio.h>
int main(){
int a = 3;
int b = 5;
printf("%d", a<b); //Output is 1 (true)
return 0;
}
```

3) Logical Operator (&&, ||, !) :

- Logical Operator, evaluates the result of a statement (or condition) consisting of a relational operator.

i) && (logical AND)

- returns true, only when both conditions(operands) are true.

ii) || (logical OR)

- returns false, only when both conditions(operands) are false.

iii) ! (logical NOT)

- return true when the condition is false and false when it is true

Condition A	Condition B	A && B (AND)	A B (OR)	!A (NOT A)
False (0)	False (0)	False (both false)	False (both false)	True (opposite of false)
False (0)	True (1)	False (both not true)	True (one true)	True (opposite of false)
True (1)	False (0)	False (both not true)	True (one true)	False (opposite of true)
True (1)	True (1)	True (both true)	True (one or both true)	False (opposite of true)

Example 7 :

```
#include <stdio.h>
int main(){
int a = 5, b= 7, c = 9;
```

```
printf("%d", ( (a<b) && (a>=c) ) );//Output is 0, since only first condition is true
printf("%d", ( (a<b) || (a>=c) ) );//Output is 1, since only second condition is false
print("%d", !(a == 5)); // Output is 0, since the condition is true
return 0;
}
```

4) Bitwise Operator(&, |, <<, >>, ~, ^) :

- Bitwise operators work **at the bit level** of the operands.
- Before performing the operation, the operands are **converted to binary (bits)**, and then the operation is done on those bits.

i) & (bitwise AND)

- Returns **1** only if both bits are 1, otherwise returns **0**.

Example 8:

```
#include <stdio.h>
int main(){
int a = 10; //1010
int b = 2; //0010
printf("%d", a&b); //Output is 2 which is 0010 in binary
return 0;
}
```

ii) | (bitwise OR)

- Returns **0** only if both bits are 0, otherwise returns **1**.

Example 9 :

```
#include <stdio.h>
int main(){
int a = 10; //1010
int b = 2; //0010
```

```
printf("%d", a|b); //Output is 10 which is 1010 in binary
y
return 0;
}
```

iii) << (left - shift)

- Shifts the bits of the left operand **to the left** by the number of positions specified by the right operand.
- Fills the emptied bits on the right with **0**.

Example 10:

```
#include <stdio.h>
int main(){
int a = 10; //001010
printf("%d", a<<2); //Output is 40 which is 101000 in binary
return 0;
}
```

iv) >> (right - shift)

- Shifts the bits of the left operand **to the right** by the number of positions specified by the right operand.
- Fills the emptied bits on the left with **0** (for unsigned numbers).

Example 11:

```
#include <stdio.h>
int main(){
int a = 10; //1010
int b = 2; //0010
printf("%d", a>>b); //Output is 2 which is 0010 in binary
y
return 0;
}
```

v) ^ (bitwise XOR)

- Returns **0** if **both bits are the same**; returns **1** if the bits are different.

Example 12:

```
#include <stdio.h>
int main(){
int a = 10; //1010
int b = 2; //0010
printf("%d", a^b); //Output is 8 which is 1000 in binary
return 0;
}
```

Example 13:

```
#include <stdio.h>
int main(){
int a = 11;
print("%d",
return 0;
}
```

5) Assignment Operator(=, +=, -=, *=, /=, %=) :

- Assigns the value of one operand or operation to the other operand

Operator	Description
=	c = a assigns the value of a to c
+=	c += a is equivalent to c = c + a
-=	c -= a is equivalent to c = c - a
*=	c *= a is equivalent to c = c * a
/=	c /= a is equivalent to c = c / a
%=	c %= a is equivalent to c = c % a

Example 14:

```
#include <stdio.h>
int main(){
int x = 20;
x += 10;
printf("x+10 = %d", x); //now x is 30
```

```
x -= 10;
printf("\nx-10 = %d", x); //now x is 20

x *= 10;
printf("\nx*10 = %d", x); //now x is 200

x /= 10;
printf("\nx/10 = %d", x); //now x is 20

x %= 10;
printf("\nx%%10 = %d", x); //now x is 0
return 0;
}
```

3. Ternary Operator:

1) Conditional Operator(? :):

- The **ternary operator** is a shortcut for simple `if-else` conditions.
- It **evaluates a condition** and returns one of two values depending on whether the condition is **true or false**.
- The **condition** is written **before the** `?` symbol.
- The expression **after** `?` runs if the condition is **true**.
- The expression **after** `:` runs if the condition is **false**.
- It helps to write **shorter and cleaner code** by replacing simple `if-else` statements.

Syntax :

(Conditional statement) ? Statement 1 : Statement 2

 ↑ ↑ ↑

 Condition Executed, Executed,

 if true if false

↑
Condition

↑
Executed,
if true

↑
Executed,
if false

Example 15:

```
#include <stdio.h>
int main(){
    int num = 13;
    (num % 2 == 0)? printf("The number is even") : printf("The number is odd");
    return 0;
}
//Since the condition is false, the output is "The number is odd"
```

Why use the ternary operator?

- Makes your code more **concise**.
- Useful when you want to **assign or print** based on a condition in a single line.

★ Conditional Statements

◆ Why use Conditional Statements?

- Conditions allow us to **make decisions** in a program.
- By using **relational** and **logical operators**, we can create expressions that return either **true (1)** or **false (0)**.
- Conditional statements help us **control the flow** of the program and instruct the machine **what to do when a condition is met or not met**.

◆ Types of Conditional Statements in C:

1 if Statement

📖 Explanation:

The `if` statement is used when you want to execute **a block of code only when a certain condition is true**. If the condition is false, the code inside `if` does not run.

📌 **Think like this:**

"If this condition is true, then do this task."

```
if(condition) {  
    // code to execute if condition is true  
}
```

✓ Example:

```
int a = 5, b = 3;  
if(a > b) {  
    printf("a is greater than b");  
}
```

 Output: `a is greater than b`

2 if - else Statement

📘 Explanation:

The `else` statement comes after an `if` and is used when the **if condition is false**. So, **either if runs or else runs**, not both.

📌 Think like this:

"If this is true, do this. Otherwise, do that."

```
if(condition) {  
    // code if true  
} else {  
    // code if false  
}
```

✓ Example:

```
int r = 2, p = 5;  
if(r > p) {  
    printf("Not equal");  
} else {
```

```
printf("Equal");  
}
```

 Output: `Equal`

3 `else if` Statement

Explanation:

When you want to check **multiple conditions**, use `else if`. If the `if` condition is false, it checks the next one. You can have many `else if` s.

Think like this:

"If not this, then maybe that."

```
if(condition1) {  
    // executes if condition1 is true  
} else if(condition2) {  
    // executes if condition2 is true  
} else {  
    // executes if none are true  
}
```

Example:

```
int a = 10, b = 13;  
if (a > b) {  
    printf("%d", a);  
} else if (a < b) {  
    printf("%d", b);  
} else {  
    printf("a and b are equal");  
}
```

 Output: `13`

4 `Nested if-else` Statement

Explanation:

A **nested if-else** means an **if** inside another **if**. It is used when **a condition depends on another condition**. The second condition is only checked **if the first is true**.

Think like this:

"If this is true, then check something else."

```
if(condition1) {  
    if(condition2) {  
        // executes only if both conditions are true  
    } else {  
        // executes if only condition1 is true  
    }  
} else {  
    // executes if condition1 is false  
}
```

Example:

```
int z;  
printf("Enter a number = ");  
scanf("%d", &z);  
  
if(z % 2 == 0) {  
    if(z % 3 == 0) {  
        printf("%d is an even number divisible by 3", z);  
    } else {  
        printf("%d is an even number but not divisible by 3", z);  
    }  
} else {  
    printf("%d is not an even number", z);  
}
```



Sample Input: 6



Output: 6 is an even number divisible by 3

switch Statement

Explanation:

The `switch` statement is used when you have **many values to compare**. Instead of using many `if-else`, we can use `switch` to make the code clean and readable.

Think like this:

"If the value is this, do this specific thing."

```
switch(expression) {  
    case value1:  
        // code  
        break;  
    case value2:  
        // code  
        break;  
    default:  
        // code if no match  
}
```

Example:

```
int option;  
printf("Enter option - ");  
scanf("%d", &option);  
  
switch(option) {  
    case 1:  
        printf("Selected option is 1");  
        break;  
    case 2:  
        printf("Selected option is 2");  
        break;  
    case 3:  
        printf("Selected option is 3");  
        break;  
    default:
```

```
printf("Invalid option");  
}
```

 Input: 2

 Output: Selected option is 2

Note:

- Always use `break` after each case to avoid fall-through.
- `default` runs if no case matches.
- We use `break` to **stop the execution** after a matching case is found. Otherwise, it would continue running other cases too (this is called **fall-through**).

Loops

◆ Why use Loops?

- Loops allow us to **execute a block of code multiple times**, either a fixed or variable number of times.
- They help in **automating repetitive tasks** and controlling program flow efficiently.
- They make the code **more efficient, clean, and shorter**, especially when doing **repetitive tasks**.
- Instead of writing the same code again and again, we can use a loop to repeat it.

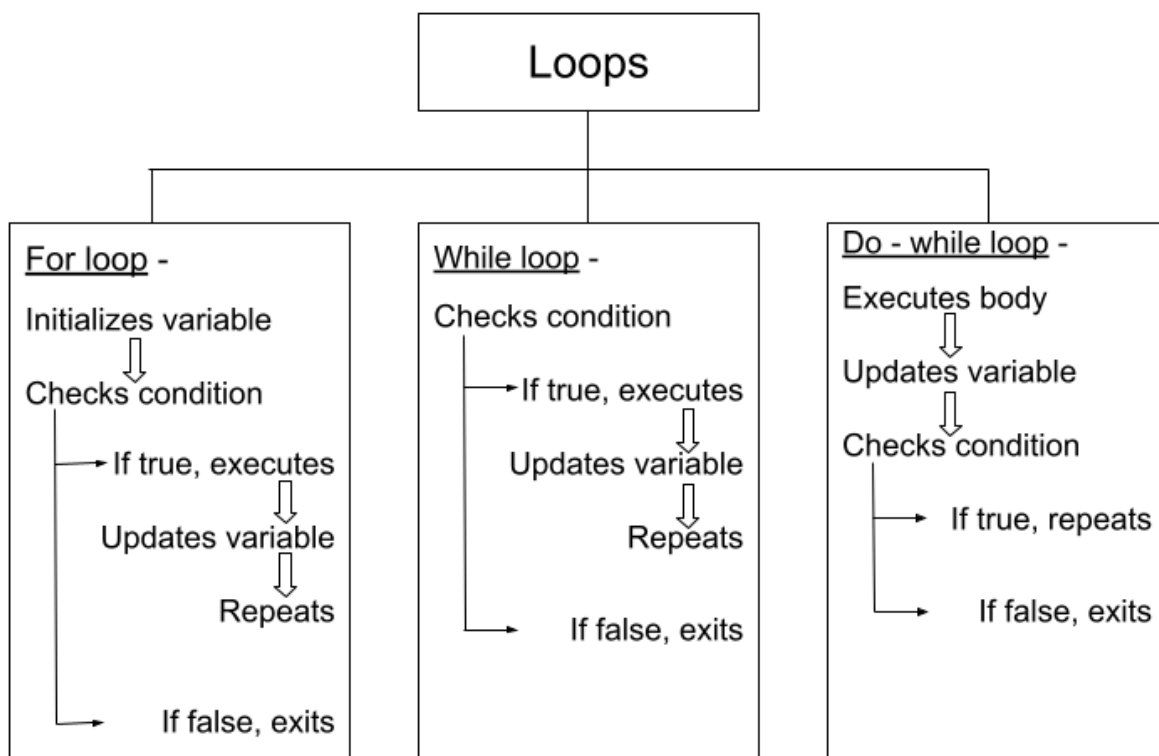
 **Think like this:**

"If you want to do something again and again, don't write it again — just loop it!"

◆ Types of Loops in C:

There are **3 primary types of loops** in C:

- 1 **while** loop
- 2 **for** loop
- 3 **do-while** loop



1 **while** loop

📘 Explanation:

- Used when the **number of iterations is not known** but we know the **condition**.
- It checks the condition **before** every iteration.

📌 Think like this:

"While this is true, keep doing this."

```
while(condition) {  
    // code block  
}
```

✓ Example:

```
#include <stdio.h>  
int main() {  
    int j = 1;  
    while(j <= 3) {  
        printf("This is a while loop.\n");  
        j++;  
    }  
    return 0;  
}
```



Output:

```
This is a while loop.  
This is a while loop.  
This is a while loop.
```

2 for loop

📘 Explanation:

- **for** loop is used when the **number of iterations is known**.
- It has **three parts**:
initialization, **condition**, and **updation** — all in one line.

📌 Think like this:

"Start from this point, repeat until this condition is true, and move forward each time."

```
for(initialization; condition; updation){  
    // code block  
}
```

✓ Example:

```
#include <stdio.h>  
int main() {  
    for(int i = 1; i <= 5; i++) {  
        printf("This is a for loop.\n");  
    }  
    return 0;  
}
```



Output:

```
This is a for loop.  
This is a for loop.  
This is a for loop.  
This is a for loop.  
This is a for loop.
```

3 do-while loop

📘 Explanation:

- Like `while`, but the condition is **checked after** executing the code block.
- It ensures the code **runs at least once**, even if the condition is false.

📌 Think like this:

"Do this once, and then keep doing it while this is true."

```
do {  
    // code block  
} while(condition);
```

✓ Example:


```
#include <stdio.h>
int main() {
    int k = 1;
    do {
        printf("This is a do-while loop.\n");
        k++;
    } while(k > 5);
    return 0;
}
```



Output:

This is a do-while loop.



Note: Even though `k > 5` is false, the loop runs **once**.



Nested Loops



Explanation:

- A **loop inside another loop** is called a **nested loop**. There can be any number of loops inside a loop.
- It is used to print patterns, work with multidimensional arrays, etc.
- For each **outer loop** iteration, the **inner loop runs completely**.



Think like this:

| "For each outer round, complete all the inner rounds."



Example:

```
#include <stdio.h>
int main() {
    int n = 3, i, j;
    for(i = 1; i <= n; i++) {
        for(j = 1; j <= n; j++) {
            printf("* ");
        }
    }
}
```

```
    printf("\n");
}
return 0;
}
```



Output:

```
* * *
* * *
* * *
```



You can also use nested while and do-while loops in the same way.

Break Statement



Explanation:

- `break` is used to **exit a loop or a switch** statement **immediately**.
- It **stops further execution** of the loop and jumps out.



Think like this:

| "I'm done here, let's get out of this loop!"



Example:

```
#include <stdio.h>
int main() {
    for(int i = 1; i <= 5; i++) {
        if(i == 3) {
            printf("Loop terminated at %d\n", i);
            break;
        }
        printf("Break Statement %d\n", i);
    }
    return 0;
}
```



Output:

```
Break Statement 1
Break Statement 2
Loop terminated at 3
```



Continue Statement



Explanation:

- `continue` skips the **current iteration** and goes to the **next iteration** of the loop.
- It is used when you **want to skip something** under a condition.



Think like this:

"Skip this time, I'll continue from next."



Example:

```
#include <stdio.h>
int main() {
    for(int j = 1; j <= 5; j++) {
        if(j == 4) {
            continue; // skips when j == 4
        }
        printf("%d ", j);
    }
    return 0;
}
```



Output:

```
1 2 3 5
```

Functions

◆ What is a Function?

A **function** is a **block of code** that performs a **specific task**. It helps us divide a program into small, **manageable**, and **reusable** parts. Instead of writing the same code again and again, we write it once inside a function and **reuse** it whenever needed.

📌 Key Points:

- Functions make the program **modular** and **organized**.
- A function **executes only when it is called**.
- It is defined **once** but can be used **multiple times**.

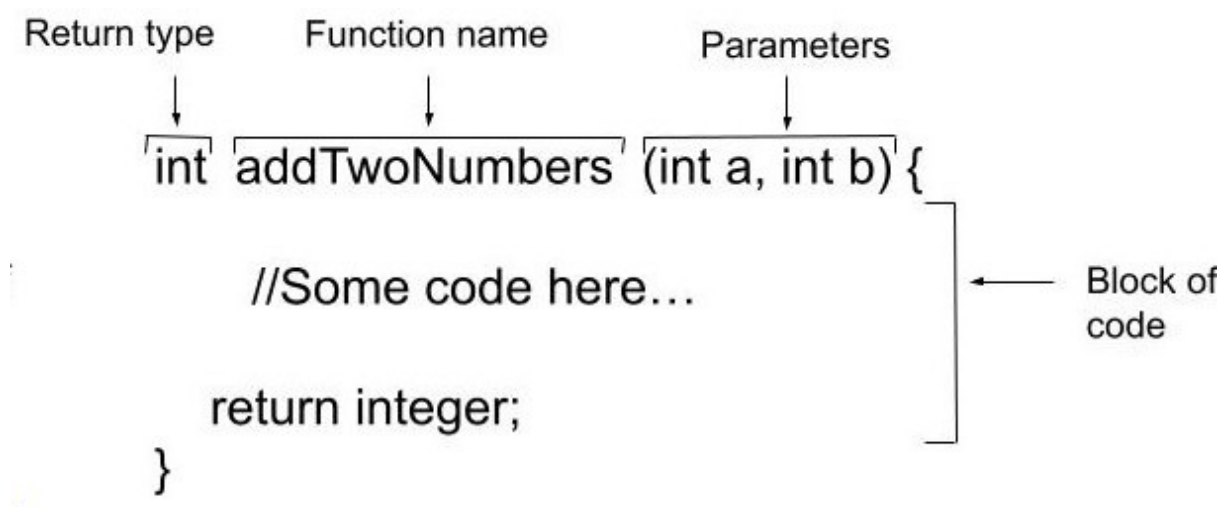
📦 Think like this:

"A function is like a machine: you give input, it processes it, and returns output (if any)."

Types of Functions in C

Functions are mainly of **two types**:

1 Return Type Function : (int , float , long , double . char)



2 Non Return Type Function : (void)

Diagram illustrating the syntax of a Non Return Type Function (void):

```
void sumOfTwoNumbers (int a, int b) {  
    //Some code here...  
}
```

Labels in the diagram:

- Function name: `sumOfTwoNumbers`
- Parameters: `(int a, int b)`
- Block of code: `{ ... }`

✓ 1) Return Type Function

📘 Definition:

- A function that **returns a value** after execution.
- The returned value must match the **data type** specified in the function definition.

🔧 Syntax:

```
returnType functionName(parameter1, parameter2, ...) {  
    // code  
    return value;  
}
```

📦 Example:

```
int addTwoNumbers(int m, int n) {  
    int sum = m + n;  
    return sum;  
}
```

📌 Explanation:

- In the **"Return type function"**, the function returns a value after execution. That value has to be of data type that is mentioned in 'Return type',
- **For example, the `addTwoNumbers` (return type is `int`) function returns the sum which is an integer.**

Real-Life Analogy:

Like a calculator: You press 5 + 6, and it gives you 11 as output.

2) Non Return Type Function

Definition:

- A function that **does not return any value**.
- `void` means "nothing".

Syntax:

```
void functionName(parameter1, parameter2, ...) {  
    // code  
}
```

Example:

```
void sumOfTwoNumbers(int k, int j) {  
    int sum = k + j;  
    printf("Sum is %d", sum);  
}
```

Explanation:

- `void` is used because we **don't want to return** anything.
- The **"Void function"** does not return any value. Hence, we write void at the beginning, which indicates that the function upon being called, does not return any value.
- **For example, the `sumOfTwoNumbers` function prints the value of sum but does not return any value.**

💡 Real-Life Analogy:

Like a speaker: You give it input and it directly plays the sound — no result is returned back.

📦 Function Name

- A function name is the identifier given to a function. It is used to call the function.
- In the given examples, `addTwoNumbers` and `sumOfTwoNumbers` are function names.

🎯 Parameters & Arguments (Function Inputs)

💎 Parameters:

- **Parameters** are the **placeholders (variables)** declared in the function **definition**.
- They represent the **inputs** that the function **expects to receive** when it is called.
- Parameters are placed inside parentheses `()` of the function.
- Any number of parameters, separated by commas can be passed in a function depending upon their usage.

📦 Example:

```
void printAlpha(char ch) {  
    printf("Alpha, %c!", ch);  
}
```

Here, `char ch` is the **parameter**.

🧠 Think like:

"Parameters are the empty containers or blanks waiting to be filled when the function is called."

◆ Arguments:

- **Arguments** are the **actual values** or **data** passed to the function when it is **called**.
- They are used to fill the parameters and are given **inside the parentheses** during a function **call**.

📦 Example:

```
int main() {  
    printAlpha('A'); // 'A' is the argument  
    return 0;  
}
```

Here, `"Aman"` is the **argument** passed to the function `greet()`.

🧠 Think like:

“Arguments are the real ingredients you add into the recipe (function) when you want to cook a dish (output).”

🚀 Calling a Function

Here's how you **call a function** from `main()`:

```
#include <stdio.h>  
  
// Return type function  
int addTwoNumbers(int m, int n) {  
    int sum = m + n;  
    return sum;  
}  
  
// Void function  
void sumOfTwoNumbers(int k, int j) {  
    int sum = k + j;  
    printf("Sum is %d\n", sum);  
}
```



```

int main() {
    int a = 5, b = 6;

    // Calling Return type function
    printf("Returned Sum is %d\n", addTwoNumbers(a, b)); // Output: 11

    // Calling Void function
    sumOfTwoNumbers(a, b); // Output: 11 (printed directly)

    return 0;
}

```

What Happens When a Function is Called?

Understanding the flow of control when a function is invoked is crucial. Here's how the execution happens step-by-step:

Step-by-Step Execution Flow:

Step 1:

Control Transfer

The moment a function is **called**, the **control of the program shifts** from `main()` (or wherever it's called) to the **function definition**.

Step 2:

Function Execution

The function **performs its specific task**. The **statements inside the function block** are executed **line by line**.

Step 3:

Return (if applicable)

- If it's a **return type function**, it **returns a value** using the `return` keyword.
- If it's a **void function**, it **does not return any value** — it simply finishes its work (e.g., printing).

Step 4:

Return to Caller

After completing the task:

- **Control goes back** to the point **just after the function call**.
- The program **continues execution** from the next statement.



Important Note:

- ◆ The execution of every C program always begins with the `main()` function.
- ◆ A function **must be defined or declared** before it's used, otherwise the compiler will throw an error.



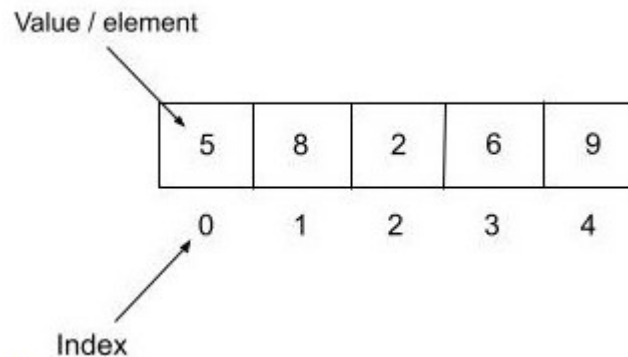
Arrays

◆ What is an Array?

- An **array** is a **linear data structure** in C that allows you to **store multiple values** of the **same data type** in a **single variable**.
- It is used when you want to store a **list of values** rather than creating individual variables for each item.
- Arrays store elements in **contiguous memory locations**, which means the elements are placed one after the other in memory.
- Each element in the array is identified by a **unique index number**, which starts from `0`.
 - So, in an array of size 5:
 - `arr[0]` is the **first element**
 - `arr[1]` is the **second element**, and so on...
- Arrays can be **single-dimensional (1D)** or **multi-dimensional (2D, 3D, etc.)**, but the most commonly used are **1D** and **2D arrays**.

◆ 1D Array (One-Dimensional Array)

- A 1D array, also known as a one-dimensional array, is a simple data structure in C that holds multiple values of the same data type in a linear form.



Memory Layout Example:

Suppose we have the following array:

```
int arr[5] = {5, 8, 2, 6, 9};
```

Memory Representation:

```
Index:  0  1  2  3  4
Values: 5  8  2  6  9
```

 Indexing starts at 0, and each index holds one element of the array.

◆ Declaring a 1D Array

```
int array1[5]; // Declaration: creates space for 5 integers
```

- `int` → data type of array elements
- `array1` → array name
- `[5]` → total number of elements (size)

◆ Declaring and Initializing Together

```
int array2[3] = {19, 17, 32}; // Declaration + Initialization
```

This means:

- `array2[0] = 19`
- `array2[1] = 17`
- `array2[2] = 32`

✅ You can also let the compiler detect size automatically:

```
int array3[] = {10, 20, 30}; // Size = 3 inferred by compiler
```

◆ Printing Values from a 1D Array

```
#include <stdio.h>
int main() {
    int array2[3] = {19, 17, 32};

    printf("Value at index 0 = %d\n", array2[0]); // 19
    printf("Value at index 1 = %d\n", array2[1]); // 17
    printf("Value at index 2 = %d\n", array2[2]); // 32

    return 0;
}
```

◆ Taking Input from User in Array

```
#include <stdio.h>
int main() {
    int size = 5;
    int arr[size];

    printf("Enter %d elements:\n", size);
    for (int i = 0; i < size; i++) {
        scanf("%d", &arr[i]);
    }
}
```

```

printf("Array elements are:\n");
for (int i = 0; i < size; i++) {
    printf("%d ", arr[i]);
}

return 0;
}

```

🧠 This program first **takes input** and then **prints all elements**.

◆ How to Find Size of an Array in C?

Size of an array:

- When the size of an array is not known to us, it can be calculated by using the `sizeof` operator. `sizeof` gives the size in bytes.
- If we want to find the number of elements in an array, we can divide the total size of the array by the size of one element (which is the byte size of the datatype of that array).

When the size is not explicitly mentioned:

```

int arr[] = {5, 6, 2, 9};
int size = sizeof(arr) / sizeof(arr[0]); // OR sizeof(int)
printf("Size of array = %d", size); // Output: 4

```

- In the above example, `sizeof( array )` gives the total size of the array in bytes, and `sizeof( array [0])` gives the size of one element. The division of these two values gives the total number of elements in the array.

💡 `sizeof(arr)` gives total memory occupied

💡 `sizeof(arr[0])` gives memory of one element (like 4 bytes for `int`)

◆ Visual Diagram

1D Array: `arr[5] = {5, 8, 2, 6, 9}`

Values: 5 8 2 6 9
Index: [0] [1] [2] [3] [4]

2D Arrays

◆ What is a 2D Array?

- A **2D array** (two-dimensional array) is like a **table or matrix** with **rows and columns**.
- It is a **collection of 1D arrays**, making it a powerful structure to **store tabular data** (like matrices or grids).
- Each element in a 2D array is accessed by **two indices** – one for **row** and one for **column**.

 Think of it like this:

```
matrix[row][column]
```

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

◆ Example of a 2D Matrix

Let's take a 3×3 matrix:

```
int matrix[3][3] = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9}  
};
```

Memory Representation:

Index:	Values:
[0][0]	1
[0][1]	2
[0][2]	3
[1][0]	4
[1][1]	5
[1][2]	6
[2][0]	7
[2][1]	8
[2][2]	9

 `matrix[0][0]` means **1st row, 1st column**, and so on.

◆ Declaring a 2D Array

```
int array1[5][5];
```

- `int` → data type of elements
- `array1` → name of array
- `[5][5]` → 5 rows and 5 columns

◆ Declaring + Initializing a 2D Array

```
int array2[3][3] = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9}  
};
```

This creates a **3×3 matrix** like:

```
1 2 3  
4 5 6  
7 8 9
```

◆ Accessing Elements in a 2D Array

You can access elements using their row and column indices:

```
array2[0][0] → 1  
array2[2][1] → 8  
array2[1][0] → 4
```

◆ Example Code: Printing Values from 2D Array

```
#include <stdio.h>  
int main() {  
    int array2[3][3] = {  
        {1, 2, 3},  
        {4, 5, 6},  
        {7, 8, 9}  
    };  
  
    printf("Value at [0][0] = %d\n", array2[0][0]); // 1  
    printf("Value at [2][1] = %d\n", array2[2][1]); // 8  
    printf("Value at [1][0] = %d\n", array2[1][0]); // 4  
  
    return 0;  
}
```

◆ Taking Input in a 2D Array from User

```
#include <stdio.h>  
int main() {  
    int arr[2][3]; // 2 rows and 3 columns  
  
    printf("Enter 6 elements:\n");  
    for (int i = 0; i < 2; i++) {  
        for (int j = 0; j < 3; j++) {  
            scanf("%d", &arr[i][j]);  
        }  
    }  
}
```



```

    }
}

printf("Matrix:\n");
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 3; j++) {
        printf("%d ", arr[i][j]);
    }
    printf("\n");
}

return 0;
}

```

✅ This program:

- Takes 6 values (2 rows × 3 columns) from the user
- Prints them in matrix form

◆ Visual Diagram of 2D Array

```

int matrix[3][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

```

📊 Representation:

Row ↓ / Col → [0] [1] [2]

[0]	1	2	3
[1]	4	5	6
[2]	7	8	9

Note: It is important to note that arrays in C are fixed in size. This means once you declare an array, you can't change its size.

Strings

- There is a 'char' data type in C which allows only one character to be stored in a variable. The value assigned to the variable has to be inside single quotes (' ').
 - When a string is stored in a char array, the array ends with a null character. To elaborate, the element at the last index of a char array is '\0', which is a null character.
 - A **string** in C is essentially a **one-dimensional array of characters** terminated by a **null character** ('\0').
 - Each character takes 1 byte in memory, and the null character is crucial to signal the **end of the string**.
 - This stores each character of the string on each index of the array.
-

◆ Declaring Strings in C

There are **two ways** to declare strings:

1 By Char Array

```
char str[] = {'c','o','d','i','n','g','A','g','e','\0'};
char str[10] = {'c','o','d','i','n','g','A','g','e'};
// '\0' is automatically added if space is available
```

👉 To print:

```
int s = sizeof(str) / sizeof(char);
for(int i = 0; i < s; i++) {
    printf("%c", str[i]);
}
```

👉 Better way using `'\0'` condition:

```
for(int i = 0; str[i] != '\0'; i++) {  
    printf("%c", str[i]);  
}
```

2 By String Literal

```
char str[] = "CodingAge";    // Automatically adds '\0'  
char str[10] = "Coding Age"; // Must fit in the size including '\0'
```

🔧 How to Declare a String

```
char stringName[size];  
  
int size = 5;  
char str[size];
```



How to Take Input

1. Character by Character (Using Loop)

```
for(int i = 0; i < size; i++) {  
    scanf(" %c", &str[i]);  
}
```

2. Using `scanf()`

```
scanf("%s", str);  
// ⚠️ Cannot read multi-word strings
```

3. Using `fgets()` ✅ (Recommended)

`fgets()` function requires three arguments -

- first, the char array, where we are going to store the string input.

- second, size of the char array(if not known, can be calculated by using ' `sizeof()` ' operator).
- `stdin`, takes input from the user through keyboard.

```
fgets(str, sizeof(str), stdin);  
// Safe input function. Reads until newline or size-1 characters.
```

4. Using `getchar()`

```
char ch = getchar();  
// Reads a single character
```



How to Print Output

1. Character by Character

```
for(int i = 0; str[i] != '\0'; i++) {  
    printf("%c", str[i]);  
}
```

2. Using `printf()`

```
printf("%s", str);
```

3. Using `puts()`

```
puts(str);  
// Adds a newline automatically
```

4. Using `putchar()`

```
putchar(str[i]);  
// Prints one character at a time
```

String Functions (`<string.h>`)

- The `<string.h>` header file contains **declarations of standard string functions** like `strlen()` , `strcpy()` , `strcat()` , and `strcmp()` .
- Without including `<string.h>` , the compiler **won't recognize these functions**, causing errors or warnings.

✓ 1. `strlen()` – Get string length

- Returns the length of the string.

```
char str[] = "Hello";  
int len = strlen(str);  
printf("Length: %d", len); // Output: 5
```

✓ 2. `strcpy()` – Copy a string

- Copy first n characters of one string to another.

```
char src[] = "Source";  
char dest[10];  
strcpy(dest, src); // dest = "Source"
```

✓ 3. `strcat()` – Concatenate (join) two strings

- Concatenates(joins) two strings.

```
char str1[20] = "Hello ";  
char str2[] = "World!";  
strcat(str1, str2); // str1 = "Hello World!"
```

✓ 4. `strcmp()` – Compare two strings

- Compares two strings. (returns the difference in ascii value of two strings)

```
char a[] = "A";  
char b[] = "B";
```

```
int result = strcmp(a, b); // Returns ASCII difference: 'A' - 'B' = -1
```

✓ 5. `strtok()` – Tokenize (split) a string

- Split the given string into tokens based on some character as a delimiter such as space, comma, dot etc.

```
char str[] = "check-in";  
char *token = strtok(str, "-");  
printf("%s", token); // Output: check
```

Pointers

◆ What is a Pointer?

A **pointer** is a special type of variable that stores the **memory address** of another variable of the **same data type**.

- Every variable in C is stored at a unique memory location (address).
- A pointer can access and manipulate the value at that address.
- Pointers enable powerful operations like:
 - Dynamic memory allocation
 - Efficient array and string manipulation
 - Function argument passing by reference
 - Complex data structure implementations (linked lists, trees, etc.)

◆ Pointer Declaration and Initialization

✓ Syntax:

```
data_type *pointer_name;
```

✓ Example:

```
int a = 5;
int *ptr = &a;
```

- `&a` gives the **address** of variable `a`.
- `ptr` is a pointer that **stores the address** of `a`.
- `*ptr` gives the **value** stored at that address (i.e., value of `a`).
- if variable '`ptr`' stores the address of variable '`a`', then variable '`ptr`' has to be a pointer. It also means that variable '`ptr`' points(→) to the location where variable '`a`' is located.
- If pointer is of int data type, the address that it stores has to be of a variable of int data type only. Same goes for pointers of other data types as well. Meaning, pointer and the variable whose address it stores have to be of same data type.
- Pointers allow us to make changes at the memory level.
- Using a pointer we can access the value of the variable that is being pointed by that pointer.
- To do this, we need to add asterisk(`*`) to the left of the pointer variable.

◆ Referencing and Dereferencing

Operator	Name	Meaning
<code>&</code>	Address-of operator	Gets the memory address of a variable
<code>*</code>	Dereference operator	Accesses the value at a memory address

✓ Example:

```
#include <stdio.h>
int main(){
    int a = 5;
    int *p = &a;
    printf("Address of a: %p\n", p); // prints address
    printf("Value of a: %d\n", *p); // prints 5
    *p = 7;
```

```
printf("Updated a: %d\n", a);    // prints 7
return 0;
}
```

◆ Size of Pointer

- Pointer size depends on the system architecture:
 - **32-bit system** → 4 bytes
 - **64-bit system** → 8 bytes

```
int *ptr;
printf("Size of pointer: %lu\n", sizeof(ptr));
```

◆ Pointer to an Array

- Pointers can be used to access and manipulate array elements efficiently.
- Pointer to an array points to the address of memory block of an array .
- We can easily move the pointer to point to each index of the array.

```
#include <stdio.h>
int main(){
    int arr[5] = {10, 20, 30, 40, 50};
    int *ptr = arr; // or &arr[0]

    for (int i = 0; i < 5; i++) {
        printf("%d ", *(ptr + i)); // or ptr[i]
    }
    return 0;
}
```

◆ Pointer to a String

```
#include <stdio.h>
int main(){
```



```

char str[20] = "Cagers";
char *ptr = str;

while(*ptr != '\0'){
    printf("%c ", *ptr);
    ptr++;
}
return 0;
}

```

✓ String Length Using Pointer:

```

#include <stdio.h>
int length(char *p) {
    int i = 0;
    while (p[i]) {
        i++;
    }
    return i;
}
int main() {
    char str[20] = "Cagers";
    int l = length(str);
    printf("Length = %d", l);
    return 0;
}

```

◆ Pointer with Function (Call by Reference)

↻ Swapping Two Numbers Using Pointer:

```

#include <stdio.h>
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

```

```
int main() {
    int a = 3, b = 4;
    swap(&a, &b);
    printf("a = %d, b = %d", a, b); // Output: a = 4, b = 3
    return 0;
}
```

◆ Double Pointer (Pointer to Pointer)

A **double pointer** stores the address of another pointer.

✓ Example:

```
#include <stdio.h>
int main() {
    int x = 10;
    int *p = &x;    // pointer to x
    int **pp = &p;  // pointer to pointer

    printf("x = %d\n", x);    // 10
    printf("*p = %d\n", *p);  // 10
    printf("***pp = %d\n", **pp); // 10
    return 0;
}
```

Types of pointer:

There are various types of pointer in C. Some of them are listed below:

- **Null pointer** : Null pointer does not store the address of any variable, hence it does not point to any location. It holds the value Null.
- **Dangling pointer** : This is a pointer pointing to a memory location that has been deleted or freed.
- **Void pointer** : Void pointer does not have a specific data type. It can store the address of a variable of any data type.

Structure and Union

1. What is a Structure?

A **Structure** is a **user-defined data type** in C that allows grouping of **different types of data items** under a single name.

◆ Key Points:

- Each element inside a structure is called a **member**.
- Members are stored in **contiguous memory locations**, but each member gets its **own memory space**.
- The total size of the structure is the **sum of the sizes** of all members (may include padding for alignment).
- The keyword `struct` is used to define structures.

◆ Useful for representing real-life entities like Person, Student, Employee, etc.

Think of a structure like a **record** or a **form** – just like a student form collects `name (string)`, `age (int)`, and `percentage (float)` together, a structure can combine these in programming.

Syntax of Structure

```
struct StructureName {  
    data_type member1;  
    data_type member2;  
    ...  
};
```

Example:

```
struct Student {  
    char name[50];  
    int rollNo;  
    float percentage;  
};
```

In this example, the structure `Student` stores the name (string), roll number (integer), and percentage (float) of a student.

◆ 2. Declaring Structure Variables

There are two ways to declare variables of a structure.

🧩 Method 1: Along with Structure Definition

```
struct Student {  
    char name[50];  
    int rollNo;  
    float percentage;  
} s1, s2;
```

🧩 Method 2: After Structure Definition

```
struct Student {  
    char name[50];  
    int rollNo;  
    float percentage;  
};  
  
int main() {  
    struct Student s1;  
}
```

🔧 3. Accessing Structure Members

We use the **dot operator (.)** to access structure members.

✅ Example:

```
struct Student s1;  
  
strcpy(s1.name, "Amit");  
s1.rollNo = 101;  
s1.percentage = 85.5;
```

```
printf("Name: %s\n", s1.name);  
printf("Roll No: %d\n", s1.rollNo);  
printf("Percentage: %.2f\n", s1.percentage);
```

4. Using Structure with Pointers

When we create a pointer to a structure, we use the **arrow operator** (**->**) to access members.

Example:

```
struct Student s1 = {"Ravi", 102, 78.5};  
struct Student *ptr = &s1;  
  
printf("Name: %s", ptr->name);
```

5. Initializing a Structure

There are several ways to initialize a structure.

Method 1: Member-wise

```
struct Student s1;  
s1.rollNo = 103;  
strcpy(s1.name, "Neha");  
s1.percentage = 88.0;
```

Method 2: All at once (Positional)

```
struct Student s2 = {"Neha", 103, 88.0};
```

Method 3: Designated Initializers (C99+)

```
struct Student s3 = {  
    .name = "Ravi",
```

```
.rollNo = 104,  
.percentage = 75.6  
};
```

6. typedef with Structure

To simplify the usage of structure variables, we can use `typedef`.

Syntax:

```
typedef struct {  
    int id;  
    char name[50];  
} Employee;  
  
Employee e1;
```

Now, we can use `Employee` directly instead of writing `struct Employee`.

7. Structure with Functions

Example: Passing Structure to Function

```
struct Student {  
    char name[50];  
    int rollNo;  
};  
  
void display(struct Student s) {  
    printf("Name: %s", s.name);  
}  
  
int main() {  
    struct Student s1 = {"Amit", 101};  
    display(s1);  
}
```

8. Array of Structures

Just like you create arrays of integers, you can also create arrays of structures to store details of multiple records.

Example:

```
struct Student {
    char name[50];
    int rollNo;
};

int main() {
    struct Student students[3];

    for (int i = 0; i < 3; i++) {
        printf("Enter name: ");
        scanf("%s", students[i].name);
        printf("Enter roll number: ");
        scanf("%d", &students[i].rollNo);
    }

    for (int i = 0; i < 3; i++) {
        printf("Name: %s, Roll: %d\n", students[i].name, students[i].rollNo);
    }
}
```

9. Nested Structure (Structure inside Structure)

Example:

```
struct Address {
    char city[30];
    int pincode;
};

struct Person {
    char name[50];
```

```
struct Address addr;
};

int main() {
    struct Person p1 = {"Ravi", {"Delhi", 110001}};
    printf("Name: %s, City: %s", p1.name, p1.addr.city);
}
```

Here, `Address` is used inside the `Person` structure.

Union

1. What is a Union?

A **Union** is a user-defined data type where **all members share the same memory**. So, **only one member** can contain a value at any time.

- Size of the union = size of the **largest member**.
- It is **memory efficient** and useful when variables are not needed simultaneously.

Syntax of Union

```
union Data {
    int i;
    float f;
    char str[20];
};
```

Example:

```
union Data d;
d.i = 10;
printf("Integer: %d\n", d.i);

d.f = 5.5;
```



```
printf("Float: %.1f\n", d.f); // Now, d.i is overwritten
```

```
strcpy(d.str, "C Language");  
printf("String: %s\n", d.str);
```

⚠ **Important:** Only the **last assigned member** will have a valid value. Previous ones get corrupted.

🧠 Key Differences: Structure vs Union

Feature	Structure	Union
Memory	Allocates memory for all members	Allocates memory for the largest member only
Access	Can access all members at once	Can access only one member at a time
Size	Sum of all members' sizes	Size of the largest member
Use Case	Group multiple values	Save memory when only one member is used at a time
Keyword	<code>struct</code>	<code>union</code>

Dynamic Memory Allocation (DMA)

◆ What is Dynamic Memory Allocation?

- **Dynamic Memory Allocation** is the process of allocating memory **during runtime** (execution time) instead of compile time.
- This helps in **efficient memory usage** as memory can be allocated as needed and freed when it's no longer required.

◆ Why is it needed?

- When we declare an array using static memory allocation, the **size is fixed** at compile time and **cannot be changed** later.
- This can lead to two problems:

- **Wastage of memory** if the array is underutilized.
- **Insufficient space** if more elements are needed than the predefined size.

◆ Difference Between Static Memory Allocation and Dynamic Memory Allocation

Feature	Static Memory Allocation (SMA)	Dynamic Memory Allocation (DMA)
Memory Allocation Time	Compile Time	Run Time
Memory Area	Stack	Heap
Size Change During Execution	Not Possible	Possible using <code>realloc()</code>
Flexibility	Rigid (fixed size)	Flexible
Memory Utilization	Can be wasteful or insufficient	More Efficient
Used In	Arrays, simple variables	Linked Lists, Trees, Stacks, Queues
Allocation Functions	No function used	<code>malloc()</code> , <code>calloc()</code> , <code>realloc()</code> , <code>free()</code>
Example	<code>int arr[10];</code>	<code>int *arr = (int*) malloc(10 * sizeof(int));</code>

◆ Example:

```
int arr[10]; // Static memory allocation
```

- Suppose we later need to store only 7 elements; the extra 3 spaces remain **unused**, wasting memory.
- Or if we need to store 15 elements later, the array cannot be resized — we need **dynamic memory** in such cases.

◆ Functions Used for DMA in C

All functions are present in the `stdlib.h` header file.

Function	Purpose
----------	---------

<code>malloc()</code>	Allocates memory but doesn't initialize it.
<code>calloc()</code>	Allocates and initializes memory to 0.
<code>realloc()</code>	Resizes previously allocated memory.
<code>free()</code>	Frees up dynamically allocated memory.

void*

- `void*` is a pointer that can hold the address of any data type (e.g., `int`, `char`, `float`, etc.).
- A pointer that can point to any type of data and needs to be cast to the appropriate type before use.

✓ 1. `malloc()` — *Memory Allocation*

◆ Features:

- Allocates a **single large block** of specified size.
- Memory is **uninitialized** (contains garbage values).
- Returns `NULL` if allocation fails.
- Returns a `void*` pointer (can be typecasted).

◆ Syntax:

```
ptr = (data_type*) malloc(size_in_bytes);
```

◆ Example:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr, n, i, sum = 0;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    ptr = (int*) malloc(n * sizeof(int)); // Allocate memory
```

```

if (ptr == NULL) {
    printf("Memory allocation failed!");
    exit(0);
}

printf("Enter array elements:\n");
for (i = 0; i < n; i++) {
    scanf("%d", ptr + i);
    sum += *(ptr + i);
}

printf("Sum = %d\n", sum);

free(ptr); // Free memory
return 0;
}

```

✓ 2. calloc() — *Contiguous Allocation*

◆ Features:

- Allocates **multiple blocks** of memory (all initialized to 0).
- Safer than `malloc()` for initializing memory.
- Also returns a `void*` pointer.

◆ Syntax:

```
ptr = (data_type*) calloc(num_elements, size_of_each_element);
```

◆ Example:

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr, n, i, sum = 0;

```

```

printf("Enter number of elements: ");
scanf("%d", &n);

ptr = (int*) calloc(n, sizeof(int)); // Allocate and initialize

if (ptr == NULL) {
    printf("Memory allocation failed!");
    exit(0);
}

printf("Enter array elements:\n");
for (i = 0; i < n; i++) {
    scanf("%d", ptr + i);
    sum += *(ptr + i);
}

printf("Sum = %d\n", sum);

free(ptr); // Free memory
return 0;
}

```

✓ 3. realloc() — *Resize Allocated Memory*

◆ Features:

- Changes the size of already allocated memory block.
- Copies old data to new block.
- Returns **NULL** if memory reallocation fails.

◆ Syntax:

```
ptr = (data_type*) realloc(ptr, new_size_in_bytes);
```

◆ Example:

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr, n1, n2, i;
    printf("Enter initial number of elements: ");
    scanf("%d", &n1);

    ptr = (int*) malloc(n1 * sizeof(int));

    if (ptr == NULL) {
        printf("Memory allocation failed!");
        exit(0);
    }

    printf("Addresses before realloc:\n");
    for (i = 0; i < n1; i++) {
        printf("%p\n", ptr + i);
    }

    printf("Enter new number of elements: ");
    scanf("%d", &n2);

    ptr = (int*) realloc(ptr, n2 * sizeof(int)); // Resize memory

    printf("Addresses after realloc:\n");
    for (i = 0; i < n2; i++) {
        printf("%p\n", ptr + i);
    }

    free(ptr); // Free memory
    return 0;
}

```

4. free() — *Deallocate Memory*

Features:

- Releases previously allocated memory back to the system.
- Prevents **memory leaks**.
- Pointer becomes **dangling**, so set it to `NULL` after freeing.

◆ Syntax:

```
free(ptr);  
ptr = NULL;
```

◆ Example:

```
#include <stdlib.h>  
int *ptr = (int*) malloc(5 * sizeof(int));  
free(ptr); // Now memory is released  
ptr = NULL; // Avoid dangling pointer
```

2D Array Using DMA

- **2D dynamic array** is a matrix (rows and columns) whose size can be decided **during program execution** instead of hardcoding it. You use it when you **don't know the size of the array in advance**.

```
#include <stdio.h>  
#include <stdlib.h>  
  
int main() {  
    int rows = 3, cols = 4;  
  
    // Step 1: Allocate memory for the array of row pointers  
    int **arr = (int **)malloc(rows * sizeof(int *));  
    if (arr == NULL) {  
        printf("Memory allocation failed!\\n");  
        return 0;  
    }  
  
    // Step 2: Allocate memory for each row
```

```

for (int i = 0; i < rows; ++i) {
    arr[i] = (int *)malloc(cols * sizeof(int));
    if (arr[i] == NULL) {
        printf("Memory allocation failed for row %d\\n", i);
        return 0;
    }
}

// Step 3: Initialize the array
int a = 1;
for (int i = 0; i < rows; ++i) {
    for (int j = 0; j < cols; j++) {
        arr[i][j] = a++;
    }
}

// Step 4: Print the array
for (int i = 0; i < rows; ++i) {
    for (int j = 0; j < cols; j++) {
        printf("%d ", arr[i][j]);
    }
    printf("\\n");
}

// Step 5: Free the allocated memory
for (int i = 0; i < rows; ++i) {
    free(arr[i]); // Free each row
}
free(arr); // Free the array of row pointers

return 0;
}

```

Explanation:

1. Allocate row pointers:

```
int **arr = malloc(rows * sizeof(int *));
```


This allocates memory for an array of rows pointers (each pointing to a row).

2. Allocate each row:

Inside the loop, for each row, we allocate memory for cols integers using

```
malloc(cols * sizeof(int));
```

3. Accessing Elements:

The elements can be accessed using `arr[i][j]` (like a regular 2D array).

4. Freeing Memory:

It's important to free each row first, and then free the array of pointers.

Advantages of Dynamic Memory Allocation:

- Dynamic Memory Allocation provides Flexibility by enabling you to allocate memory as per the actual data size and change it dynamically, unlike static allocation, which is fixed.
- It also provides **efficient usage of memory** as it optimizes memory usage by allocating memory only when required and releasing it when no longer needed. This dynamic management of memory prevents wastage and improves overall program efficiency.

File Handling

File handling is an essential concept in C programming that allows a program to **store data permanently** and **interact with files** stored on the disk. Unlike variables (which lose their data once the program ends), file handling ensures data is saved and can be retrieved later.

1. What is a File?

A **file** is a collection of related data stored on a storage device. It is used to save data **permanently** for later use.

Types of Files:

- **Text Files:** Human-readable format (e.g., `.txt`, `.csv`)
- **Binary Files:** Machine-readable format (e.g., `.bin`, `.exe`)

2. What is File Handling?

File handling in C allows you to:

- **Creating files:** Making a new file to store data.
- **Reading files:** Fetching data from a file.
- **Writing files:** Adding or updating content in a file.
- **Closing files:** Releasing system resources tied to the file after operations.

This is done using a group of standard functions from `<stdio.h>`.

3. File Modes in C

C uses file modes to define the **purpose of opening a file** (read/write/append).

Use `FILE *fopen(const char *filename, const char *mode);`

Mode	Operation	If File Exists	If File Doesn't Exist
"r"	Read-only	Opens the file	Returns NULL
"w"	Write (clears existing content)	Clears and writes	Creates a new file
"a"	Append (write to the end)	Appends to file	Creates a new file
"r+"	Read and write	Opens the file	Returns NULL
"w+"	Read and write (clears content)	Clears and writes	Creates a new file
"a+"	Read and append	Appends and reads	Creates a new file

4. Basic File Operations

Creating & Opening a File

```
FILE *file = fopen("file.txt", "w"); // Creates or overwrites file
```

Writing to a File

```
fprintf(file, "Hello, World!");    // Write formatted string
fputc('A', file);                  // Write single character
fputs("C programming", file);      // Write string
```

Reading from a File

```
fscanf(file, "%s", str);           // Read formatted data
fgetc(file);                       // Read single character
fgets(str, 100, file);             // Read a line
```

Closing a File

```
fclose(file);                      // Always close the file after us
```

4. Basic File Operations in C

Working with files in C involves using a pointer of type `FILE*`, provided by the standard I/O library (`stdio.h`). Below are the most common operations:

1. Creating & Opening a File

```
FILE *file = fopen("file.txt", "w");
```

Explanation:

- `"w"` mode creates a new file or **overwrites** an existing file.
- `fopen()` returns a pointer to the file. If it fails, it returns `NULL`.

2. Opening a File in Read Mode

```
FILE *file = fopen("file.txt", "r");
```

Explanation:

- `"r"` mode opens an existing file for **reading only**.
- If the file does **not exist**, it returns `NULL`.

* 3. Opening a File in Append Mode

```
FILE *file = fopen("file.txt", "a");
```

✓ Explanation:

- `"a"` mode opens the file for **writing at the end** (append).
- If the file does not exist, it will be **created**.

* 4. Writing to a File

```
fprintf(file, "Hello, World!\n"); // Write a formatted string
fputc('A', file);                // Write a single character
fputs("C programming", file);    // Write a string
```

✓ Explanation:

- `fprintf()` is like `printf()`, but outputs to a file.
- `fputc()` writes a single character.
- `fputs()` writes a null-terminated string.

* 5. Reading from a File

```
char str[100];

fscanf(file, "%s", str); // Read a formatted string (until space)
fgetc(file);            // Read a single character
fgets(str, 100, file);  // Read an entire line
```

✓ Explanation:

- `fscanf()` is like `scanf()`, reads formatted data.
- `fgetc()` reads one character at a time.
- `fgets()` reads a whole line (up to given size or newline).

* 6. Appending Data to a File

```
FILE *file = fopen("file.txt", "a");  
fputs("Adding more text.\n", file);
```

✅ **Explanation:**

- Appends data at the **end** of the file without erasing existing content.

✳️ 7. Closing a File

```
fclose(file);
```

✅ **Explanation:**

- Always close files after operations to **save changes** and **free resources**.

👉 Example Program:

```
#include <stdio.h>  
  
int main() {  
    FILE *file;  
    char str[100];  
    char ch;  
  
    // Creating & Opening a file in write mode ("w")  
    file = fopen("file.txt", "w");  
    if (file == NULL) {  
        printf("Error opening file for writing.\n");  
        return 1;  
    }  
  
    // Writing to the file  
    fprintf(file, "Hello, World!\n");    // Write formatted string  
    fputc('A', file);                    // Write single character  
    fputs("\nC programming", file);     // Write string  
  
    // Close the file after writing  
    fclose(file);
```

```

// Opening the file in read mode ("r")
file = fopen("file.txt", "r");
if (file == NULL) {
    printf("Error opening file for reading.\n");
    return 1;
}

// Reading from the file
printf("Reading characters one by one:\n");
while ((ch = fgetc(file)) != EOF) {
    putchar(ch); // Print each character read from the file
}

// Reset file pointer to the beginning
rewind(file);

// Reading a line using fgets
if (fgets(str, sizeof(str), file) != NULL) {
    printf("\nReading first line with fgets(): %s", str);
}

// Close the file after reading
fclose(file);

return 0;
}

```

Explanation:

- `w+` mode allows reading & writing, and clears the file if it exists.
- Always check if `fp == NULL` to handle errors.
- `fclose()` releases the resources tied with the file.

6. `fseek()` — Move the File Pointer

The `fseek()` function lets you move the file pointer to a specific location within the file.



Syntax:

```
int fseek(FILE *stream, long int offset, int origin);
```

Parameters :

- **pointer** : It is the pointer that points to the FILE that we need to modify.
- **offset** : It is the number of characters or bytes, where the position of the file pointer needs to be shifted relative to the current position to determine the new position.
- **position** : It is the position from where the offset is added. Position defines the point with respect to which the file pointer needs to be moved. It has three values:
 - SEEK_END : It denotes the end of the file.
 - SEEK_SET : It denotes starting of the file
 - SEEK_CUR : It denotes the file pointer's current position.

Return value :

- It returns zero if successful, or else it returns a non-zero value.

Example: Using fseek() and ftell()

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *fp = fopen("file.txt", "w+");
    if (fp == NULL) {
        printf("File not found!\n");
        exit(1);
    }

    fprintf(fp, "abcdefghijk");

    printf("Current Position: %ld\n", ftell(fp)); // 11
```

```
fseek(fp, 4, SEEK_END); // Move 4 bytes after end
printf("New Position: %ld\n", ftell(fp));    // 15

fclose(fp);
return 0;
}
```

📌 **ftell()** returns the current position of the file pointer (in bytes).

📌 **fseek()** is useful for random access (like moving to a specific record in a binary file).
